

Project Update #1: Gym Equipment Object Detection using YOLOv7

Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors

Team Members: Varun Gazala | Mohit Raiyani | Jatinkumar Nabhoya

University of New Haven | Spring 2026

Paper: [arXiv:2207.02696](https://arxiv.org/abs/2207.02696)

Repository: [WongKinYiu/yolov7](https://github.com/WongKinYiu/yolov7)

Task Overview

This notebook implements multi-class object detection of **5 gym equipment categories** (dumbbell, barbell, kettlebell, resistance_band, pull_up_bar) — none of which appear in the COCO dataset that YOLOv7 was pretrained on.

Contents

1. Environment Setup & Installation
 2. Dataset Collection & Processing
 3. Pretrained Model Performance (Inference)
 4. Transfer Learning / Model Adaptation
 5. Summary & Next Steps
-

✓ 1. Environment Setup & Installation

```
# Clone YOLOv7 repository and install dependencies
import os

if not os.path.exists("yolov7"):
    !git clone https://github.com/WongKinYiu/yolov7.git
%cd yolov7

# Install deps individually (YOLOv7 requirements.txt pins numpy<1.24 whi
!pip install -q matplotlib numpy opencv-python Pillow PyYAML requests sc
```

```
# Download pretrained weights (COCO) – curl works on both macOS and Linux
if not os.path.exists("yolov7.pt"):
    !curl -L -o yolov7.pt https://github.com/WongKinYiu/yolov7/releases/
    print("Downloaded yolov7.pt (COCO pretrained)")
else:
    print("yolov7.pt already exists")
```

```
/Users/jatinnabhoya/Desktop/UNH/Semester 3/Deep Learning /Project/phase 1
yolov7.pt already exists
```

```
# — Imports —
import sys, os, random, shutil, glob, yaml, warnings, copy
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import torchvision
import cv2
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from pathlib import Path
from collections import Counter, defaultdict
warnings.filterwarnings('ignore')

# YOLOv7 imports (repo must be cloned first)
sys.path.insert(0, '.')
from models.experimental import attempt_load
from utils.general import non_max_suppression, check_img_size, xywh2xyxy
from utils.torch_utils import select_device
from utils.datasets import letterbox
from utils.loss import ComputeLoss
```

```
# Device setup
device = select_device('') # auto: CUDA > MPS > CPU
print(f"\nUsing device: {device}")
print(f"PyTorch version: {torch.__version__}")
print(f"Torchvision version: {torchvision.__version__}")
if device.type == 'cuda':
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"GPU Memory: {torch.cuda.get_device_properties(0).total_mem /
```

```
Using device: cpu
PyTorch version: 2.10.0
Torchvision version: 0.25.0
```

```
# — Configuration —
NUM_CLASSES = 5
CLASS_NAMES = ['dumbbell', 'barbell', 'kettlebell', 'resistance_band', '
```

```

CLASS_COLORS = [
    (0.85, 0.32, 0.32),    # dumbbell - red
    (0.30, 0.69, 0.29),    # barbell - green
    (0.22, 0.46, 0.82),    # kettlebell - blue
    (0.93, 0.69, 0.13),    # resistance_band - gold
    (0.58, 0.26, 0.81),    # pull_up_bar - purple
]
IMG_SIZE = 640
BATCH_SIZE = 8
RANDOM_SEED = 42
MAX_VIS_BATCHES = 5  # Max minibatches to visualize

# Reproducibility
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)
torch.manual_seed(RANDOM_SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(RANDOM_SEED)

DATASET_ROOT = '../dataset'
print("Configuration loaded successfully.")
print(f"  Classes ({NUM_CLASSES}): {CLASS_NAMES}")
print(f"  Image size: {IMG_SIZE}x{IMG_SIZE}")
print(f"  Batch size: {BATCH_SIZE}")

```

```

Configuration loaded successfully.
  Classes (5): ['dumbbell', 'barbell', 'kettlebell', 'resistance_band', '
  Image size: 640x640
  Batch size: 8

```

✓ 2. Dataset Collection & Processing

2.1 Dataset Directory Structure

We organize the dataset in the standard YOLO format expected by the YOLOv7 repository:

```

dataset/
├── images/
│   ├── train/      (80%)
│   ├── val/        (10%)
│   └── test/       (10%)
├── labels/
│   ├── train/
│   └── val/

```

```
|   └─ test/
└─ data.yaml
```

Annotation format: YOLO TXT — one `.txt` file per image, each line: `class_id x_center y_center width height` (normalized 0–1)

Class ID	Class Name	Description
0	dumbbell	Free weights for arm/shoulder exercises
1	barbell	Long weighted bar for squats/deadlifts
2	kettlebell	Cast iron ball with handle
3	resistance_band	Elastic bands for stretching/resistance
4	pull_up_bar	Horizontal overhead bar on a rack

Note: All 5 classes are absent from the COCO dataset (80 classes) that YOLOv7 was pretrained on, making this a valid transfer learning task.

```
# — Create directory structure —
for split in ['train', 'val', 'test']:
    os.makedirs(f'{DATASET_ROOT}/images/{split}', exist_ok=True)
    os.makedirs(f'{DATASET_ROOT}/labels/{split}', exist_ok=True)
os.makedirs(f'{DATASET_ROOT}/images/all', exist_ok=True)
os.makedirs(f'{DATASET_ROOT}/labels/all', exist_ok=True)

# Create data.yaml configuration
data_config = {
    'train': os.path.abspath(f'{DATASET_ROOT}/images/train'),
    'val': os.path.abspath(f'{DATASET_ROOT}/images/val'),
    'test': os.path.abspath(f'{DATASET_ROOT}/images/test'),
    'nc': NUM_CLASSES,
    'names': CLASS_NAMES
}

with open(f'{DATASET_ROOT}/data.yaml', 'w') as f:
    yaml.dump(data_config, f, default_flow_style=False)

print("Dataset directory structure created.")
print(f"data.yaml saved to {DATASET_ROOT}/data.yaml")
print(f"\ndata.yaml contents:")
with open(f'{DATASET_ROOT}/data.yaml', 'r') as f:
    print(f.read())
```

Dataset directory structure created.
data.yaml saved to ../dataset/data.yaml

data.yaml contents:
names:
- dumbbell
- barbell
- kettlebell

```

- resistance_band
- pull_up_bar
nc: 5
test: /Users/jatinnabhoya/Desktop/UNH/Semester 3/Deep Learning /Project/p
train: /Users/jatinnabhoya/Desktop/UNH/Semester 3/Deep Learning /Project/
val: /Users/jatinnabhoya/Desktop/UNH/Semester 3/Deep Learning /Project/ph

```

✓ 2.3 Dataset Partitioning (80% Train / 10% Val / 10% Test)

We partition the dataset with approximate stratification based on the primary class in each image. This ensures balanced class representation across all splits.

```

from sklearn.model_selection import train_test_split

def partition_dataset(root_dir, train_r=0.8, val_r=0.1, test_r=0.1, seed
    """Split dataset into train/val/test with stratification."""
    assert abs(train_r + val_r + test_r - 1.0) < 1e-6

    images_all = sorted(glob.glob(os.path.join(root_dir, 'images', 'all'

# Check if already split
train_existing = glob.glob(os.path.join(root_dir, 'images', 'train',
if len(train_existing) > 0 and len(images_all) == 0:
    print("Dataset already partitioned. Skipping.")
    for s in ['train', 'val', 'test']:
        n = len(glob.glob(os.path.join(root_dir, 'images', s, '*.*)
        print(f" {s}: {n} images")
    return

if len(images_all) == 0:
    raise ValueError("No images found in images/all/")

# Get primary class per image for stratification
primary = []
for p in images_all:
    lbl = os.path.join(root_dir, 'labels', 'all', Path(p).stem + '.t
    if os.path.exists(lbl):
        with open(lbl) as f:
            lines = f.readlines()
            primary.append(int(lines[0].split()[0]) if lines else 0)
    else:
        primary.append(0)

# Two-stage split
train_imgs, temp_imgs, train_c, temp_c = train_test_split(
    images_all, primary, test_size=(val_r + test_r),
    random_state=seed, stratify=primary)

```

```

rel_test = test_r / (val_r + test_r)
val_imgs, test_imgs, _, _ = train_test_split(
    temp_imgs, temp_c, test_size=rel_test,
    random_state=seed, stratify=temp_c)

# Copy files
splits = {'train': train_imgs, 'val': val_imgs, 'test': test_imgs}
for split, img_list in splits.items():
    for ip in img_list:
        stem = Path(ip).stem
        ext = Path(ip).suffix
        shutil.copy2(ip, os.path.join(root_dir, 'images', split, f'{
src_lbl = os.path.join(root_dir, 'labels', 'all', f'{stem}.t
if os.path.exists(src_lbl):
    shutil.copy2(src_lbl, os.path.join(root_dir, 'labels', s

# Statistics
print(f"{'Split':<8} {'Images':>8} {'Pct':>8}")
print("-" * 26)
total = len(images_all)
for s, il in splits.items():
    print(f"{s:<8} {len(il):>8} {len(il)/total*100:>7.1f}%")
print(f"{'Total':<8} {total:>8} {'100.0%':>8}")

# Per-class distribution
print(f"\n{'Class':<20} {'Train':>6} {'Val':>6} {'Test':>6}")
print("-" * 42)
counts = {}
for s, il in splits.items():
    cc = Counter()
    for ip in il:
        lbl = os.path.join(root_dir, 'labels', 'all', Path(ip).stem
        if os.path.exists(lbl):
            with open(lbl) as f:
                for line in f:
                    cc[int(line.split()[0])] += 1
    counts[s] = cc
for cid, cn in enumerate(CLASS_NAMES):
    print(f"{cn:<20} {counts['train'].get(cid,0):>6} "
          f"{counts['val'].get(cid,0):>6} {counts['test'].get(cid,0)}

partition_dataset(DATASET_ROOT, seed=RANDOM_SEED)

```

Split	Images	Pct
train	162	79.8%
val	20	9.9%
test	21	10.3%
Total	203	100.0%

Class	Train	Val	Test
-------	-------	-----	------

dumbbell	75	6	8
barbell	91	19	11
kettlebell	24	2	8
resistance_band	25	3	3
pull_up_bar	32	5	3

✓ 2.4 Custom Dataset Class

We define a `GymEquipmentDataset` class that:

- Loads images from the YOLO directory structure
- Parses `.txt` annotation files in YOLO format
- Applies augmentations (training) or just resize + normalize (val/test)
- Returns data compatible with both YOLOv7's loss function and mAP evaluation

```
class GymEquipmentDataset(Dataset):
    """Custom dataset for gym equipment detection in YOLO format.

    Expects:
        images_dir: path to images folder (e.g., dataset/images/train/)
        labels_dir: path to labels folder (e.g., dataset/labels/train/)
        img_size: target image size (default 640)
        augment: whether to apply training augmentations

    Returns:
        img: tensor (3, H, W) normalized [0,1]
        targets: tensor (N, 6) -> [batch_idx, class, x_center, y_center,
        path: image file path
    """
    def __init__(self, images_dir, labels_dir, img_size=640, augment=False):
        self.images_dir = images_dir
        self.labels_dir = labels_dir
        self.img_size = img_size
        self.augment = augment

        # Collect image paths
        self.img_files = sorted(glob.glob(os.path.join(images_dir, '*.*')))
        self.img_files = [f for f in self.img_files
                           if f.lower().endswith(('.jpg', '.jpeg', '.png'))]

        assert len(self.img_files) > 0, f"No images found in {images_dir}"

        # Pre-load all labels
        self.labels = []
        for img_path in self.img_files:
            stem = Path(img_path).stem
            lbl_path = os.path.join(labels_dir, f'{stem}.txt')
```

```

        if os.path.exists(lbl_path):
            with open(lbl_path) as f:
                lines = f.readlines()
                lbl = []
                for line in lines:
                    parts = line.strip().split()
                    if len(parts) == 5:
                        cls_id = int(parts[0])
                        coords = [float(x) for x in parts[1:]]
                        lbl.append([cls_id] + coords)
                self.labels.append(np.array(lbl, dtype=np.float32) if
                                   else np.zeros((0, 5), dtype=np.flo
            else:
                self.labels.append(np.zeros((0, 5), dtype=np.float32))

    print(f" Loaded {len(self.img_files)} images from {images_dir}"
          total_objs = sum(len(l) for l in self.labels)
    print(f" Total annotations: {total_objs}")

def __len__(self):
    return len(self.img_files)

def __getitem__(self, idx):
    # Load image (BGR -> RGB)
    img_path = self.img_files[idx]
    img = cv2.imread(img_path)
    assert img is not None, f"Failed to load {img_path}"
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    labels = self.labels[idx].copy()
    h0, w0 = img.shape[:2]

    # Apply augmentations or just resize
    if self.augment:
        img, labels = self._augment(img, labels)

    # Letterbox resize to target size
    img, ratio, (dw, dh) = letterbox(img, self.img_size, auto=False,

    # Normalize pixel values: [0,255] -> [0,1]
    img = img.astype(np.float32) / 255.0

    # Convert HWC -> CHW
    img = np.transpose(img, (2, 0, 1))
    img = np.ascontiguousarray(img)
    img = torch.from_numpy(img)

    # Prepare targets: [class, x, y, w, h] (already normalized)
    if len(labels) > 0:
        targets = torch.from_numpy(labels).float()
    else:

```



```

        targets = torch.zeros((0, 5), dtype=torch.float32)

    return img, targets, img_path

def _augment(self, img, labels):
    """Apply training augmentations per YOLOv7 paper (bag-of-freebie
    h, w = img.shape[:2]

    # 1. HSV color jitter (hue=0.015, sat=0.7, val=0.4)
    if random.random() < 0.5:
        img = self._hsv_augment(img, h_gain=0.015, s_gain=0.7, v_gai

    # 2. Random horizontal flip
    if random.random() < 0.5:
        img = np.fliplr(img).copy()
        if len(labels) > 0:
            labels[:, 1] = 1.0 - labels[:, 1] # flip x_center

    # 3. Random scale ( $\pm 50\%$ )
    if random.random() < 0.3:
        scale = random.uniform(0.5, 1.5)
        new_h, new_w = int(h * scale), int(w * scale)
        img = cv2.resize(img, (new_w, new_h))

    return img, labels

def _hsv_augment(self, img, h_gain=0.015, s_gain=0.7, v_gain=0.4):
    """HSV color space augmentation as used in YOLOv7."""
    r = np.random.uniform(-1, 1, 3) * [h_gain, s_gain, v_gain] + 1
    hue, sat, val = cv2.split(cv2.cvtColor(img, cv2.COLOR_RGB2HSV))
    dtype = img.dtype

    x = np.arange(0, 256, dtype=r.dtype)
    lut_hue = ((x * r[0]) % 180).astype(dtype)
    lut_sat = np.clip(x * r[1], 0, 255).astype(dtype)
    lut_val = np.clip(x * r[2], 0, 255).astype(dtype)

    im_hsv = cv2.merge((cv2.LUT(hue, lut_hue), cv2.LUT(sat, lut_sat)
    return cv2.cvtColor(im_hsv, cv2.COLOR_HSV2RGB)

print("GymEquipmentDataset class defined.")

GymEquipmentDataset class defined.

```

2.5 Normalization & Augmentation Summary

Per the YOLOv7 paper (Trainable Bag-of-Freebies approach):

Normalization:

- Pixel values scaled from $[0, 255] \rightarrow [0.0, 1.0]$
- Bounding box coordinates in YOLO format are already normalized to $[0, 1]$
- Batch Normalization layers within the network handle internal feature normalization

Training Augmentations (Bag-of-Freebies):

- **HSV jitter:** hue ± 0.015 , saturation ± 0.7 , value ± 0.4
- **Random horizontal flip** ($p=0.5$)
- **Random scale** ($0.5-1.5\times$)
- **Letterbox resize** to 640×640 with preserved aspect ratio

Validation/Test: Only letterbox resize + pixel normalization (no augmentation)

✓ 2.6 DataLoaders & Dataset Statistics

```
def collate_fn(batch):
    """Custom collate for variable-length annotations.

    Prepends batch index to each target tensor so the loss function
    knows which image each annotation belongs to.
    """
    imgs, targets, paths = zip(*batch)
    imgs = torch.stack(imgs, 0)

    # Add batch index column
    new_targets = []
    for i, t in enumerate(targets):
        if len(t) > 0:
            batch_col = torch.full((len(t), 1), i, dtype=torch.float32)
            new_targets.append(torch.cat([batch_col, t], dim=1))

    if new_targets:
        targets = torch.cat(new_targets, 0)
    else:
        targets = torch.zeros((0, 6), dtype=torch.float32)

    return imgs, targets, list(paths)

# — Create Datasets —
print("Loading datasets...")
train_img_dir = f'{DATASET_ROOT}/images/train'
train_lbl_dir = f'{DATASET_ROOT}/labels/train'
val_img_dir = f'{DATASET_ROOT}/images/val'
val_lbl_dir = f'{DATASET_ROOT}/labels/val'
test_img_dir = f'{DATASET_ROOT}/images/test'
test_lbl_dir = f'{DATASET_ROOT}/labels/test'
```

```

train_dataset = GymEquipmentDataset(train_img_dir, train_lbl_dir, IMG_SIZE)
val_dataset = GymEquipmentDataset(val_img_dir, val_lbl_dir, IMG_SIZE, au)
test_dataset = GymEquipmentDataset(test_img_dir, test_lbl_dir, IMG_SIZE, au)

# — Create DataLoaders —
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
                           collate_fn=collate_fn, num_workers=2,
                           # pin_memory=True
                           )
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
                        collate_fn=collate_fn, num_workers=2,
                        # pin_memory=True
                        )
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
                        collate_fn=collate_fn, num_workers=2,
                        # pin_memory=True
                        )

print(f"\n{'Dataset Summary':^50}")
print(f"{'Split':<10} {'Images':>8} {'Batches':>10}")
print("-" * 30)
print(f"{'Train':<10} {len(train_dataset):>8} {len(train_loader):>10}")
print(f"{'Val':<10} {len(val_dataset):>8} {len(val_loader):>10}")
print(f"{'Test':<10} {len(test_dataset):>8} {len(test_loader):>10}")

```

Loading datasets...

```

Loaded 162 images from ../dataset/images/train
Total annotations: 247
Loaded 20 images from ../dataset/images/val
Total annotations: 35
Loaded 21 images from ../dataset/images/test
Total annotations: 33

```

```

=====Dataset Summary=====
Split      Images      Batches
-----
Train      162         21
Val        20          3
Test       21          3

```

```

# — Visualize sample augmented images —
fig, axes = plt.subplots(2, 4, figsize=(20, 10))
fig.suptitle('Sample Training Images with Augmentation & Ground Truth',
             zorder=0)

for i in range(8):
    ax = axes[i // 4][i % 4]
    img, targets, path = train_dataset[i]

    # Convert CHW -> HWC for display
    img_np = img.permute(1, 2, 0).numpy()
    ax.imshow(img_np)

```

```

# Draw ground truth boxes
# targets: [class, x_center, y_center, w, h] (normalized)
h, w = img_np.shape[:2]
for t in targets:
    cls_id = int(t[0])
    xc, yc, bw, bh = t[1] * w, t[2] * h, t[3] * w, t[4] * h
    x1, y1 = xc - bw/2, yc - bh/2
    rect = patches.Rectangle((x1, y1), bw, bh, linewidth=2,
                             edgecolor=CLASS_COLORS[cls_id], facecolor='none')
    ax.add_patch(rect)
    ax.text(x1, y1 - 3, CLASS_NAMES[cls_id], fontsize=7, color='white',
            bbox=dict(boxstyle='round,pad=0.2', facecolor=CLASS_COLORS[cls_id]))

ax.set_title(f'Image {i}', fontsize=10)
ax.axis('off')

plt.tight_layout()
plt.savefig('../sample_augmented_images.png', dpi=150, bbox_inches='tight')
plt.show()
print("Sample augmented images saved to ../sample_augmented_images.png")

```

Sample augmented images saved to ../sample_augmented_images.png

✓ 3. Pretrained Model Performance

3.1 Load Pretrained YOLOv7 (COCO — 80 classes)

We load the official YOLOv7 weights trained on MS COCO (80 object categories). Our 5 gym equipment classes (dumbbell, barbell, kettlebell, resistance_band, pull_up_bar) are **not** in the COCO dataset, so the pretrained model cannot detect them directly.

```

# — Load pretrained YOLOv7 (COCO) —
model_pretrained = attempt_load('yolov7.pt', map_location=device)
model_pretrained.eval()

# Model information
total_params = sum(p.numel() for p in model_pretrained.parameters())
trainable_params = sum(p.numel() for p in model_pretrained.parameters())

print(f"{'Pretrained Model Summary':^50}")
print(f"  Architecture: YOLOv7")
print(f"  Pretrained on: MS COCO (80 classes)")
print(f"  Total parameters: {total_params:,}")
print(f"  Trainable parameters: {trainable_params:,}")
print(f"  Input size: {IMG_SIZE}x{IMG_SIZE}")
print(f"  Number of classes (COCO): {model_pretrained.module.nc if hasattr(model_pretrained.module, 'nc') else 0}")

```

```

print(f"\nCOCO class names (first 20):")
coco_names = model_pretrained.module.names if hasattr(model_pretrained,
if isinstance(coco_names, dict):
    coco_names = [coco_names[i] for i in range(len(coco_names))]
for i in range(min(20, len(coco_names))):
    print(f" {i:>3d}: {coco_names[i]}")
print(f" ... ({len(coco_names)} total)")

```

```

Fusing layers...
RepConv.fuse_repvgg_block
RepConv.fuse_repvgg_block
RepConv.fuse_repvgg_block
=====Pretrained Model Summary=====
Architecture: YOLOv7
Pretrained on: MS COCO (80 classes)
Total parameters: 36,905,341
Trainable parameters: 36,905,341
Input size: 640x640
Number of classes (COCO): 80

COCO class names (first 20):
0: person
1: bicycle
2: car
3: motorcycle
4: airplane
5: bus
6: train
7: truck
8: boat
9: traffic light
10: fire hydrant
11: stop sign
12: parking meter
13: bench
14: bird
15: cat
16: dog
17: horse
18: sheep
19: cow
... (80 total)

```

3.2 Inference on Training Data — Visualize 1st Sample per Minibatch

Since the pretrained model knows COCO classes (not gym equipment), we expect it to either:

- Detect nothing, or
- Misclassify objects as visually similar COCO categories

This demonstrates **why fine-tuning is needed**.

```
# Fix: Recreate DataLoaders with num_workers=0 (required for macOS + Jup
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=
                        collate_fn=collate_fn, num_workers=0)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False
                        collate_fn=collate_fn, num_workers=0)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False
                        collate_fn=collate_fn, num_workers=0)
print("DataLoaders recreated with num_workers=0")
```

DataLoaders recreated with num_workers=0

```
def visualize_inference(model, dataloader, class_names, title_prefix="Pr
                        max_batches=5, conf_thres=0.25, iou_thres=0.45):
    """Run inference and visualize the 1st sample of each minibatch.

    Args:
        model: YOLOv7 model in eval mode
        dataloader: DataLoader to iterate
        class_names: list of class names for the model's predictions
        title_prefix: prefix for plot titles
        max_batches: maximum number of batches to visualize
        conf_thres: confidence threshold for NMS
        iou_thres: IoU threshold for NMS
    """
    model.eval()

    fig, axes = plt.subplots(1, min(max_batches, len(dataloader)),
                            figsize=(5 * min(max_batches, len(dataloader)),
                                     5 * min(max_batches, len(dataloader))))
    if not isinstance(axes, np.ndarray):
        axes = [axes]

    fig.suptitle(f'{title_prefix} Model - 1st Sample per Minibatch',
                 fontsize=14, fontweight='bold')

    with torch.no_grad():
        for batch_idx, (imgs, targets, paths) in enumerate(dataloader):
            if batch_idx >= max_batches:
                break

            imgs_dev = imgs.to(device)

            # Run inference
            pred = model(imgs_dev)[0]
            pred = non_max_suppression(pred, conf_thres=conf_thres, iou_

            # Take first sample
            img_0 = imgs[0].permute(1, 2, 0).cpu().numpy()
```

```

det_0 = pred[0] # detections for first image

ax = axes[batch_idx]
ax.imshow(img_0)

# Draw GT boxes (green dashed)
gt_mask = targets[:, 0] == 0 # batch index 0
gt_boxes = targets[gt_mask]
h, w = img_0.shape[:2]
for gt in gt_boxes:
    cls_id = int(gt[1])
    xc, yc, bw, bh = gt[2]*w, gt[3]*h, gt[4]*w, gt[5]*h
    x1, y1 = xc - bw/2, yc - bh/2
    rect = patches.Rectangle((x1, y1), bw, bh, linewidth=2,
                             edgecolor='lime', facecolor='n
    ax.add_patch(rect)
    ax.text(x1, y1 - 3, f'GT: {CLASS_NAMES[cls_id]}', fontsi
            bbox=dict(boxstyle='round,pad=0.2', facecolor='g

# Draw predictions (red solid)
if det_0 is not None and len(det_0) > 0:
    for *xyxy, conf, cls_pred in det_0:
        x1, y1, x2, y2 = [v.cpu().item() for v in xyxy]
        cls_id = int(cls_pred.cpu().item())
        score = conf.cpu().item()
        rect = patches.Rectangle((x1, y1), x2-x1, y2-y1, lin
                                edgecolor='red', facecolor
        ax.add_patch(rect)
        name = class_names[cls_id] if cls_id < len(class_nam
        ax.text(x1, y2 + 12, f'{name}: {score:.2f}', fontsi
                bbox=dict(boxstyle='round,pad=0.2', facecolo
        ax.set_title(f'Batch {batch_idx} - {len(det_0)} detectio
    else:
        ax.set_title(f'Batch {batch_idx} - No detections', fonts

ax.axis('off')

# Legend
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], color='lime', linewidth=2, linestyle='--', labe
    Line2D([0], [0], color='red', linewidth=2, label='Prediction')
]
fig.legend(handles=legend_elements, loc='lower center', ncol=2, font
plt.tight_layout(rect=[0, 0.05, 1, 0.95])
plt.savefig(f'../{title_prefix.lower().replace(" ", "_")}_inference.
plt.show()
print(f"Visualization saved.")

# Run pretrained inference on training data
coco_names_list = coco_names if isinstance(coco_names, list) else [coco_

```

```
visualize_inference(model_pretrained, train_loader, coco_names_list,
                    title_prefix="Pretrained COCO", max_batches=MAX_VIS_
```

Visualization saved.

✓ 4. Transfer Learning / Model Adaptation

4.1 Replace Output Layers (80 COCO classes → 5 Gym Equipment classes)

The YOLOv7 detection head (`IDetect`) has 3 output scales (P3, P4, P5), each with 3 anchors. For each anchor, the output has $(nc + 5)$ values: $[x, y, w, h, obj_conf, class_1, \dots, class_nc]$.

Change: `nc` from 80 → 5

- Original output channels per scale: $(80 + 5) \times 3 = 255$
- New output channels per scale: $(5 + 5) \times 3 = 30$

We replace the final Conv2d layers in the detection head while keeping all backbone weights frozen.

```
# — Modify detection head for 5 classes —
model = attempt_load('yolov7.pt', map_location=device)

# Get the detection layer (last layer)
detect = model.model[-1]
print(f"Detection layer type: {type(detect).__name__}")
print(f"Original nc: {detect.nc}")
print(f"Number of anchors per scale: {detect.na}")
print(f"Number of output scales: {len(detect.m)}")
print(f"\nOriginal detection head Conv2d layers:")
for i, m in enumerate(detect.m):
    print(f"  Scale {i}: {m}")

# Replace detection head
new_nc = NUM_CLASSES
na = detect.na # anchors per scale (3)
new_no = new_nc + 5 # outputs per anchor

for i, m in enumerate(detect.m):
    in_ch = m.in_channels
    # Create new Conv2d with correct output channels
    detect.m[i] = nn.Conv2d(in_ch, new_no * na, kernel_size=m.kernel_size,
                           stride=m.stride, padding=m.padding).to(device)
```



```

# Update model attributes
detect.nc = new_nc
detect.no = new_no
model.nc = new_nc
model.names = CLASS_NAMES

print(f"\n{'Modified Detection Head':^50}")
print(f"  New nc: {detect.nc}")
print(f"  New outputs per anchor: {detect.no}")
print(f"  New total output channels: {new_no * na}")
print(f"\nModified detection head Conv2d layers:")
for i, m in enumerate(detect.m):
    print(f"  Scale {i}: {m}")

# Count parameters
total = sum(p.numel() for p in model.parameters())
head_params = sum(p.numel() for p in detect.m.parameters())
print(f"\nTotal model parameters: {total:,}")
print(f"Detection head parameters (new, random): {head_params:,}")
print(f"Backbone parameters (pretrained): {total - head_params:,}")

```

```

Fusing layers...
RepConv.fuse_repvgg_block
RepConv.fuse_repvgg_block
RepConv.fuse_repvgg_block
Detection layer type: Detect
Original nc: 80
Number of anchors per scale: 3
Number of output scales: 3

Original detection head Conv2d layers:
  Scale 0: Conv2d(256, 255, kernel_size=(1, 1), stride=(1, 1))
  Scale 1: Conv2d(512, 255, kernel_size=(1, 1), stride=(1, 1))
  Scale 2: Conv2d(1024, 255, kernel_size=(1, 1), stride=(1, 1))

=====Modified Detection Head=====
  New nc: 5
  New outputs per anchor: 10
  New total output channels: 30

Modified detection head Conv2d layers:
  Scale 0: Conv2d(256, 30, kernel_size=(1, 1), stride=(1, 1))
  Scale 1: Conv2d(512, 30, kernel_size=(1, 1), stride=(1, 1))
  Scale 2: Conv2d(1024, 30, kernel_size=(1, 1), stride=(1, 1))

Total model parameters: 36,501,466
Detection head parameters (new, random): 53,850
Backbone parameters (pretrained): 36,447,616

```

✓ 4.2 Baseline Evaluation — mAP (Without Training)

We evaluate the modified model **without any training** to establish a baseline. Since the detection head has randomly initialized weights, we expect:

- **mAP ≈ 0** (near-random predictions)
- **High loss values** (model hasn't learned our classes)

This baseline will be compared against post-training results in the next update.

```
def calculate_map(model, dataloader, num_classes, conf_thres=0.001, iou_
    """Calculate mAP@0.5 and mAP@0.5:0.95 for the model on a dataset.

    Returns dict with mAP metrics and per-class AP.
    """
    from torchmetrics.detection import MeanAveragePrecision

    metric = MeanAveragePrecision(
        iou_type='bbox',
        iou_thresholds=None, # default: 0.5:0.05:0.95
    )

    model.eval()
    total_gt = 0
    total_pred = 0

    with torch.no_grad():
        for batch_idx, (imgs, targets, paths) in enumerate(dataloader):
            imgs_dev = imgs.to(device)

            # Forward pass
            pred = model(imgs_dev)[0]
            pred = non_max_suppression(pred, conf_thres=conf_thres, iou_

            batch_size = imgs.shape[0]

            for i in range(batch_size):
                # Predictions for image i
                det = pred[i]
                preds_dict = {
                    'boxes': torch.zeros((0, 4)),
                    'scores': torch.zeros(0),
                    'labels': torch.zeros(0, dtype=torch.long),
                }

                if det is not None and len(det) > 0:
                    preds_dict['boxes'] = det[:, :4].cpu()
                    preds_dict['scores'] = det[:, 4].cpu()
                    preds_dict['labels'] = det[:, 5].long().cpu()
                    total_pred += len(det)

            # Ground truth for image i
```

```

gt_mask = targets[:, 0] == i
gt = targets[gt_mask]

h, w = IMG_SIZE, IMG_SIZE
gt_boxes_xyxy = torch.zeros((0, 4))
gt_labels = torch.zeros(0, dtype=torch.long)

if len(gt) > 0:
    # Convert YOLO format (xc, yc, w, h) -> xyxy
    gt_xywh = gt[:, 2:6].clone()
    gt_xyxy = xywh2xyxy(gt_xywh)
    gt_xyxy[:, [0, 2]] *= w
    gt_xyxy[:, [1, 3]] *= h
    gt_boxes_xyxy = gt_xyxy
    gt_labels = gt[:, 1].long()
    total_gt += len(gt)

target_dict = {
    'boxes': gt_boxes_xyxy.cpu(),
    'labels': gt_labels.cpu(),
}

metric.update([preds_dict], [target_dict])

results = metric.compute()

return {
    'mAP_50': results['map_50'].item(),
    'mAP_50_95': results['map'].item(),
    'total_gt': total_gt,
    'total_pred': total_pred,
}

# — Evaluate on Training Data —
print("Evaluating modified model on TRAINING data (no training done)...")
train_metrics = calculate_map(model, train_loader, NUM_CLASSES, device=device)

print("\nEvaluating modified model on VALIDATION data (no training done)")
val_metrics = calculate_map(model, val_loader, NUM_CLASSES, device=device)

# Display results
print(f"\n{'Baseline Evaluation Results (NO TRAINING)':>60}")
print(f"\n{'Metric':<25} {'Training':>15} {'Validation':>15}")
print("-" * 57)
print(f"{'mAP@0.5':<25} {train_metrics['mAP_50']:>15.4f} {val_metrics['mAP_50']:>15.4f}")
print(f"{'mAP@0.5:0.95':<25} {train_metrics['mAP_50_95']:>15.4f} {val_metrics['mAP_50_95']:>15.4f}")
print(f"{'Total GT objects':<25} {train_metrics['total_gt']:>15d} {val_metrics['total_gt']:>15d}")
print(f"{'Total predictions':<25} {train_metrics['total_pred']:>15d} {val_metrics['total_pred']:>15d}")
print(f"\nNote: mAP ≈ 0 is expected since the detection head has random weights")
print("This serves as the BASELINE for comparison after training.")

```

Evaluating modified model on TRAINING data (no training done)...

Evaluating modified model on VALIDATION data (no training done)...

=====Baseline Evaluation Results (NO TRAINING)=====

Metric	Training	Validation
mAP@0.5	0.0000	0.0005
mAP@0.5:0.95	0.0000	0.0001
Total GT objects	247	35
Total predictions	48600	6000

Note: mAP ≈ 0 is expected since the detection head has random weights. This serves as the BASELINE for comparison after training.

✓ 4.3 Average Loss Calculation (Training & Validation)

We compute the YOLOv7 loss function which has three components:

1. **Box loss** — How well predicted boxes align with ground truth (CIoU loss)
2. **Objectness loss** — Confidence that an object exists in a cell (BCE loss)
3. **Classification loss** — Correct class prediction (BCE loss)

```
# — Attach YOLOv7 default hyperparameters to model (required for Compu
model.hyp = {
    'box': 0.05,      # box loss gain
    'obj': 0.7,       # obj loss gain
    'cls': 0.3,       # cls loss gain
    'cls_pw': 1.0,    # cls BCELoss positive_weight
    'obj_pw': 1.0,    # obj BCELoss positive_weight
    'anchor_t': 4.0,  # anchor-multiple threshold
    'fl_gamma': 0.0,  # focal loss gamma
    'label_smoothing': 0.0,
}
model.gr = 1.0 # giou loss ratio
print("Hyperparameters attached to model.")
```

Hyperparameters attached to model.

```
def calculate_avg_loss(model, dataloader, device):
    """Calculate average loss over a dataset using YOLOv7's ComputeLoss.

    Returns dict with total loss and component breakdown.
    """
    model.train() # Must be in train mode for loss computation

    # Initialize loss function
```

```

compute_loss = ComputeLoss(model)

total_loss = 0.0
total_box_loss = 0.0
total_obj_loss = 0.0
total_cls_loss = 0.0
num_batches = 0

with torch.no_grad():
    for batch_idx, (imgs, targets, paths) in enumerate(dataloader):
        imgs = imgs.to(device)
        targets = targets.to(device)

        if len(targets) == 0:
            continue

        # Forward pass (train mode returns list of tensors)
        pred = model(imgs)

        # Compute loss
        loss, loss_items = compute_loss(pred, targets)

        if not torch.isfinite(loss):
            continue

        total_loss += loss.item()
        total_box_loss += loss_items[0].item()
        total_obj_loss += loss_items[1].item()
        total_cls_loss += loss_items[2].item()
        num_batches += 1

model.eval()

if num_batches == 0:
    return {'total': float('nan'), 'box': float('nan'),
            'obj': float('nan'), 'cls': float('nan'), 'batches': 0}

return {
    'total': total_loss / num_batches,
    'box': total_box_loss / num_batches,
    'obj': total_obj_loss / num_batches,
    'cls': total_cls_loss / num_batches,
    'batches': num_batches
}

# — Calculate losses —
print("Computing average loss on TRAINING data...")
train_loss = calculate_avg_loss(model, train_loader, device)

print("Computing average loss on VALIDATION data...")
val_loss = calculate_avg_loss(model, val_loader, device)

```

```

print(f"\n{'Average Loss (Before Training – Baseline)':^60}")
print(f"\n{'Loss Component':<20} {'Training':>15} {'Validation':>15}")
print("-" * 52)
print(f"{'Box Loss':<20} {train_loss['box']:>15.4f} {val_loss['box']:>15.4f}")
print(f"{'Objectness Loss':<20} {train_loss['obj']:>15.4f} {val_loss['obj']:>15.4f}")
print(f"{'Classification Loss':<20} {train_loss['cls']:>15.4f} {val_loss['cls']:>15.4f}")
print("-" * 52)
print(f"{'Total Loss':<20} {train_loss['total']:>15.4f} {val_loss['total']:>15.4f}")
print(f"\n{'Batches processed':<20} {train_loss['batches']:>15d} {val_loss['batches']:>15d}")
print(f"\nNote: High loss values are expected since the model has not been trained")

```

Computing average loss on TRAINING data...
Computing average loss on VALIDATION data...

=====Average Loss (Before Training – Baseline)=====

Loss Component	Training	Validation
Box Loss	0.0839	0.1158
Objectness Loss	3.4091	3.4193
Classification Loss	0.4729	0.6367
Total Loss	30.6379	28.0519
Batches processed	21	3

Note: High loss values are expected since the model has not been trained

4.4 Visualize 1st Sample per Minibatch — Predictions vs Ground Truth

We visualize the modified model's predictions (with random detection head weights) alongside the ground truth annotations. Predictions will be poor — this is the expected baseline.

```

# — Visualize modified model predictions —
model.eval()
visualize_inference(model, train_loader, CLASS_NAMES,
                    title_prefix="Modified Baseline (No Training)",
                    max_batches=MAX_VIS_BATCHES, conf_thres=0.01, iou_th

```

Visualization saved.

```

# — Detailed visualization: side-by-side GT vs Prediction —
model.eval()

```

```

fig, axes = plt.subplots(MAX_VIS_BATCHES, 2, figsize=(14, 5 * MAX_VIS_BATCHES))
fig.suptitle('Ground Truth (Left) vs Model Predictions (Right) – No Trai
              fontsize=16, fontweight='bold')

with torch.no_grad():
    for batch_idx, (imgs, targets, paths) in enumerate(train_loader):
        if batch_idx >= MAX_VIS_BATCHES:
            break

        imgs_dev = imgs.to(device)
        pred = model(imgs_dev)[0]
        pred = non_max_suppression(pred, conf_thres=0.01, iou_thres=0.45)

        img_0 = imgs[0].permute(1, 2, 0).cpu().numpy()
        h, w = img_0.shape[:2]

        # Left: Ground Truth
        ax_gt = axes[batch_idx][0] if MAX_VIS_BATCHES > 1 else axes[0]
        ax_gt.imshow(img_0)
        ax_gt.set_title(f'Batch {batch_idx} – Ground Truth', fontsize=11)

        gt_mask = targets[:, 0] == 0
        gt_boxes = targets[gt_mask]
        for gt in gt_boxes:
            cls_id = int(gt[1])
            xc, yc, bw, bh = gt[2]*w, gt[3]*h, gt[4]*w, gt[5]*h
            x1, y1 = xc - bw/2, yc - bh/2
            color = CLASS_COLORS[cls_id] if cls_id < len(CLASS_COLORS) else
            rect = patches.Rectangle((x1, y1), bw, bh, linewidth=2,
                                     edgecolor=color, facecolor='none')
            ax_gt.add_patch(rect)
            ax_gt.text(x1, y1 - 4, CLASS_NAMES[cls_id], fontsize=8, color=
                       bbox=dict(boxstyle='round,pad=0.2', facecolor=col
ax_gt.axis('off')

        # Right: Predictions
        ax_pr = axes[batch_idx][1] if MAX_VIS_BATCHES > 1 else axes[1]
        ax_pr.imshow(img_0)

        det = pred[0]
        if det is not None and len(det) > 0:
            ax_pr.set_title(f'Batch {batch_idx} – {len(det)} Predictions')
            for *xyxy, conf, cls_p in det:
                x1, y1, x2, y2 = [v.cpu().item() for v in xyxy]
                cls_id = int(cls_p.cpu().item())
                score = conf.cpu().item()
                color = CLASS_COLORS[cls_id] if cls_id < len(CLASS_COLORS) else
                rect = patches.Rectangle((x1, y1), x2-x1, y2-y1, linewidth=2,
                                         edgecolor=color, facecolor='no
            ax_pr.add_patch(rect)
            name = CLASS_NAMES[cls_id] if cls_id < len(CLASS_NAMES)

```

```

        ax_pr.text(x1, y1 - 4, f'{name}: {score:.3f}', fontsize=
                    bbox=dict(boxstyle='round,pad=0.2', facecolor
else:
    ax_pr.set_title(f'Batch {batch_idx} - No Predictions', fonts
ax_pr.axis('off')

plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.savefig('../baseline_gt_vs_pred.png', dpi=150, bbox_inches='tight')
plt.show()
print("Side-by-side visualization saved to ../baseline_gt_vs_pred.png")

```

Side-by-side visualization saved to ../baseline_gt_vs_pred.png

✓ 5. Summary & Next Steps

Summary of Results

Dataset

- **5 custom classes** of gym equipment (dumbbell, barbell, kettlebell, resistance_band, pull_up_bar)
- None of the 5 classes exist in COCO (80 classes), validating the transfer learning task
- **200 images** (synthetic placeholder) with YOLO-format annotations
- Partitioned: **80% train / 10% val / 10% test** with stratification
- Augmentation pipeline implements YOLOv7's bag-of-freebies: HSV jitter, random flip, random scale, letterbox resize
- Normalization: pixel values $[0,255] \rightarrow [0,1]$; bounding boxes already normalized in YOLO format

Pretrained Model (COCO)

- YOLOv7 pretrained on COCO detected COCO-class objects or nothing on our gym equipment images
- This confirms that **fine-tuning is necessary** for our custom classes

Transfer Learning Baseline (No Training)

- Detection head replaced: $80 \rightarrow 5$ classes (output channels $255 \rightarrow 30$)
- **Baseline [mAP@0.5](#) ≈ 0** — expected with randomly initialized detection head
- **High loss values** — model has not learned to detect our classes yet
- These baseline metrics will be compared against post-training results

Next Steps (Project Update #2)

1. **Replace synthetic data** with real annotated gym equipment images
2. **Train the model** — fine-tune the detection head (and optionally backbone) using our training set
3. **Hyperparameter tuning** — learning rate, warmup, weight decay per the paper
4. **Evaluate** — compare post-training mAP and loss against this baseline
5. **Full augmentation** — implement mosaic and mixup augmentation during training

End of Project Update #1

Varun Gazala | Mohit Raiyani | Jatinkumar Nabhoya

University of New Haven — Spring 2026